

附录 A

参考资料

我们已在每一章的末尾提供了一些额外的资源列表，建议读者进一步学习更多关于该章节涵盖的主题。本书的每一章都提供了对大多数读者最有价值的材料。本附录中的参考资料是对那些材料的进一步扩展。我们要感谢这些研究和技术报告的原始作者。这些材料对于那些打算对特定主题进行更深度研究的读者来说，十分重要。

A.1 第 1 章：为什么使用并行计算

- Amdahl, Gene M. “Validity of the single processor approach to achieving large scale computing capabilities.” Proceedings of the April 18–20, 1967, Spring Joint Computer Conference. (1967):483–48. <https://doi.org/10.1145/1465482.1465560>.
- Flynn, Michael J. “Some Computer Organizations and Their Effectiveness.” In *IEEE Transactions on Computers*, Vol. C-21, no. 9 (September, 1972): 948–960.
- Gustafson, John L. “Reevaluating Amdahl’s Law.” In *Communications of the ACM*, Vol. 31, no. 5 (May, 1988):532–533. <http://doi.acm.org/10.1145/42411.42415>.
- Horowitz, M., Labonte, F., and Rupp, K., et al. “Microprocessor Trend Data.” Accessed February 20, 2021. <https://github.com/karlrupp/microprocessor-trend-data>.

A.2 第 2 章：规划并行化

- CMake. <https://cmake.org/>.

A.3 第 3 章：性能极限与分析

工具:

- Empirical Roofline Toolkit (ERT). <https://bitbucket.org/berkeleylab/cs-roofline-toolkit>.
- 4Intel® Advisor. <https://software.intel.com/en-us/advisor>.

- likwid. <https://github.com/RRZE-HPC/likwid>.
- STREAM download. <https://github.com/jeffhammond/Stream.git>.
- Valgrind. <http://valgrind.org/>.

文章:

- McCalpin, J. D. “STREAM: Sustainable Memory Bandwidth in High Performance Computers.” Accessed February 20, 2021. <https://www.cs.virginia.edu/stream/>.
- Peise, Elmar. “Performance Modeling and Prediction for Dense Linear Algebra.” arXiv:1706.01341 (June, 2017). Preprint: <https://arxiv.org/abs/1706.01341>.
- Williams, S. W., D. Patterson, et. al. “The Roofline Model: A pedagogical tool for auto-tuning kernels on multicore architectures.” In *Hot Chips, A Symposium on High Performance Chips*, Vol. HC20 (August 10, 2008).

A.4 第4章：数据设计和性能模型

资源:

- Data-oriented design. <https://github.com/dbartolini/data-oriented-design>

文章和书籍:

- Bird, R. “Performance Study of Array of Structs of Arrays.” Los Alamos National Lab (LANL). Paper in preparation.
- Garimella, Rao, and Robert W. Robey. “A Comparative Study of Multi-material Data Structures for Computational Physics Applications,” no. LA-UR-16-23889.
- Los Alamos National Lab (LANL) (January, 2017).
- Hennessy, John L., and David A. Patterson. *Computer architecture: A Quantitative Approach*. 5th ed. San Francisco, CA, USA: Morgan Kaufmann, 2011.
- Hofmann, Johannes, Jan Eitzinger, and Dietmar Fey. “Execution-Cache-Memory Performance Model: Introduction and Validation.” arXiv:1509.03118 (March, 2017). Preprint: <https://arxiv.org/abs/1509.03118>.
- Hollman, David, Bryce Lebach, H. Carter Edwards, et al. “mdspan in C++: A Case Study in the Integration of Performance Portable Features into International Language Standards.” *IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)* (November, 2019):60–70.
- Treibig, Jan, and Georg Hager. “Introducing a performance model for bandwidth-limited loop kernels.” *International Conference on Parallel Processing and Applied Mathematics* (May, 2009):615–624.

A.5 第 5 章：并行算法与模式

- Ahrens, Peter, Hong Diep Nguyen, and James Demmel. “Efficient Reproducible Floating Point Summation and BLAS.” In *EECS Department, University of California, Berkeley, Technical Report*, No. UCB/EECS-2015-229 (December, 2015).
- Alcantara, Dan A., Andrei Sharf, Fatemeh Abbasinejad, et al. “Real-time parallel hashing on the GPU.” In *ACM Transactions on Graphics (TOG)*, Vol. 28, no. 5 (December, 2009):154.
- Anderson, Alyssa. “Achieving Numerical Reproducibility in the Parallelized Floating Point Dot Product.” (April, 2014). https://digitalcommons.csbsju.edu/honors_theses/30/.
- Blelloch, Guy E. “Scans as primitive parallel operations.” In *IEEE Transactions on computers*, Vol. 38, no. 11 (November, 1989):1526–1538.
- Blelloch, Guy E. *Vector models for data-parallel computing*. Cambridge, MA, USA: The MIT Press, 1990.
- Chapp, Dylan, Travis Johnston, and Michela Taufer. “On the Need for Reproducible Numerical Accuracy through Intelligent Runtime Selection of Reduction Algorithms at the Extreme Scale.” 2015 IEEE International Conference on Cluster Computing (October, 2015):166–175.
- Cleveland, Mathew A., Thomas A. Brunner, et al. “Obtaining identical results with double precision global accuracy on different numbers of processors in parallel particle Monte Carlo simulations.” In *Journal of Computational Physics*, Vol. 251 (October, 2013):223–236.
- Harris, Mark, Shubhabrata Sengupta, and John D. Owens. “Parallel Prefix Sum (Scan) with CUDA.” In *GPU Gems 3*, no. 39 (April, 2007):851-876.
- Lessley, Brenton. “Data-Parallel Hashing Techniques for GPU Architectures.” In *Eurographics Conference on Visualization (EuroVis)*, Vol. 37, no. 3 (July, 2018).

A.6 第 8 章：MPI：并行骨干

- Hoefler, Torsten, and Jesper Larsson Traff. “Sparse collective operations for MPI.” 2009 IEEE International Symposium on Parallel & Distributed Processing (July, 2009):18.
- Thakur, Rajeev, and William Gropp. “Test suite for evaluating performance of multithreaded MPI communication.” In *Parallel Computing*, Vol. 35, no. 12 (December, 2009):608–617.

A.7 第 9 章：GPU 架构及概念

- Yang, Charlene, Thorsten Kurth, and Samuel Williams. “Hierarchical Roofline analysis for GPUs: Accelerating performance optimization for the NERSC-9 Perlmutter system.” In *Concurrency and Computation: Practice and Experience* (November, 2019). <https://doi.org/10.1002/cpe.5547>.

A.8 第 10 章: GPU 编程模型

- CUDA Toolkit Documentation. “Compute Capabilities.” CUDA C++ Programming Guide, v11.2.1 (NVIDIA Corporation, 2021). <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#compute-capabilities>.

A.9 第 12 章: GPU 语言: 深入了解基础知识

- Harris, Mark. "Optimizing Parallel Reduction in CUDA." (NVIDIA Corporation). <https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf>.

A.10 第 13 章: GPU 配置分析及工具

- BBC News. “Indonesia tsunami: How a volcano can be the trigger.” BBC Global News Ltd (December, 2018). <http://mng.bz/y92d>.

A.11 第 14 章: 关联性: 与 kernel 休战

- Broquedis, François, Jérôme Clet-Ortega, et al. “hwloc: A Generic Framework for Managing Hardware Affinities in HPC Applications.” Proceedings of the 18th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP2010). IEEE Computer Society Press (February, 2010): 180–186. <https://ieeexplore.ieee.org/document/5452445>.
- Hewlett Packard Enterprise, Original process placement program, xthi.c. CLE User Application Placement Guide (CLE 5.2.UP04) S-2496, pg 87. <http://mng.bz/MgWB>.
- “OpenMP Application Programming Interface,” v5.0. OpenMP Architecture Review Board (November, 2018). <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5.0.pdf>.
- Samuel K. Gutiérrez, “Adaptive Parallelism for Coupled, Multithreaded Message-Passing Programs.” (December, 2018). <https://www.cs.unm.edu/~samuel/publications/2018/skgutierrez-dissertation.pdf>.
- Samuel K. Gutiérrez, Davis, Kei, et al. “Accommodating Thread-Level Heterogeneity in Coupled Parallel Applications.” Proceedings of the IEEE International Parallel and Distributed Processing Symposium (May, 2017). <https://github.com/lanl/libquo/blob/master/docs/publications/quo-ipdps17.pdf>.
- Squyres, Jeff. “Process Placement.” (September, 2014). Accessed February 20, 2021. <https://github.com/open-mpi/mpi/wiki/ProcessPlacement>.

- Treibig, J., G. Hager and G. Wellein. “LIKWID: A lightweight performanceoriented tool suite for x86 multicore environments.” arXiv:1004.4431(June, 2010). Preprint: <http://arxiv.org/abs/1004.4431>.

A.12 第 16 章：并行环境的文件操作

工具：

- BeeGFS (The leading parallel file system). <https://www.beegfs.io/c/>.
- Lustre[®]. OpenSFS and EOFS. <http://lustre.org>.
- The OrangeFS Project. <http://www.orangefs.org>.
- Panasas PanFS Parallel File System. <https://www.panasas.com/panfs-architecture/panfs/>.

文章和书籍：

- Gropp, William. “Lecture 33: More on MPI I/O Best practices for parallel IO and MPI-IO hints.” Accessed February 20, 2021. <http://wgropp.cs.illinois.edu/courses/cs598-s15/lectures/lecture33.pdf>.
- Mendez, Sandra, Sebastian Lührs, et al. “Best Practice Guide—Parallel I/O.” Accessed February 20, 2021. https://prace-ri.eu/wp-content/uploads/Best-Practice-Guide_Parallel-IO.pdf.
- Thakur, Rajeev, Ewing Lusk, and William Gropp. *Users guide for ROMIO: A highperformance,portable MPI-IO implementation*. ANL/MCS-TM-234. Artonne, IL, USA: Argonne National Laboratory (October, 1997).
- Thakur, Rajeev, William Gropp, and Ewing Lusk. “Data sieving and collective I/O in ROMIO.” Proceedings. Frontiers’ 99. Seventh Symposium on the Frontiers of Massively Parallel Computation (February, 1999):182–189.

A.13 第 17 章：用于编写优质代码的工具和资源

- Stepanov, Evgeniy, and Konstantin Serebryany. “MemorySanitizer: fast detector of uninitialized memory use in C++.” 2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO) (February, 2015):46–55.

B.1 第 1 章：为什么使用并行计算

1. 日常生活中并行操作的例子包括多车道高速公路、班级注册队列和邮件投递。当然，还有很多其他例子。

2. 很难看透营销和炒作，找到真正的技术规格。大多数设备，包括手持设备，都具有多核处理器和至少一个集成图形处理器。除了非常旧的硬件外，台式机和笔记本电脑都能提供一些向量计算功能。

3. 多指令、多数据(MIMD)、分布式数据、pipeline 并行和具有专用队列的无序执行。

4. 在单个计算节点上进行线程化和向量化。 $4\text{MB} \times 1024 = 4\text{GB}$ 。但是一次处理 16 个图像，只需要 64MB，在集群的每个节点(工作站)上都低于 1GB。串行运行的时间为 10 分钟 $\times$ 1000，约为 167 小时，16 个核的并行计算时间约为 10.4 分钟。向量化可将这一过程缩短到 5 分钟以下。如果需求增加 10 倍，则需要 100 分钟。这样操作可能没问题，但也可能是时候考虑消息传递或分布式计算了，从而进一步提升性能。

5. 300W/130W。需要快 2.3 倍才会被认为是更节能的。

B.2 第 2 章：规划并行化

1. 准备步骤如下。

- 使用 Git 建立版本控制系统
- 创建一组具有已知结果的测试
- 通过测试套件中的测试，运行内存正确性工具
- 分析硬件和你的应用程序
- 为敏捷管理策略制定下一步计划

2. 答案：使用 CTest 创建测试，因为 CTest 可以通过命令来检测任何错误状态，所以可以通过构建指令进行测试。有时安装 CTest 文件会从权限中去掉可执行的部分，并导致测试失败，并且不

提供明确的错误信息。为了避免这种情况，我们可以添加一个测试来检测 CTest 脚本是否可以执行。在下面的代码中，\$0 是带有完整路径的 CTest 脚本，这样它就可以用于 out-of-tree 构建。

- 在 CMakeLists.txt 中添加如下内容：

```
enable_testing()

add_test(NAM Emake WORKING_DIRECTORY ${CMAKE_BINARY_DIRECTORY}
         COMMAND${CMAKE_CURRENT_SOURCE_DIR}/build.ctest)
```

- 在 build.ctest 文件中添加如下内容：

```
#!/bin/sh
if [-x$0]
then
    echo"PASSED-isexecutable"
else
    echo"Failed-ctestscriptisnotexecutable"
    exit-1
Fi
```

3. 你可以通过修改或添加以下代码行来修复代码清单 2-2 中的内存错误：

```
4   int ipos=0, ival;
7   for (int i = 0; i<10; i++){ iarray[i] = ipos; }
8   for (int i = 0; i<10; i++){
11  free(iarray);
```

4. 请自己选择应用程序并完成练习。

B.3 第 3 章：性能极限与分析

1. 请自己选择并完成练习。
2. 请自己选择并完成练习。
3. 请自己选择并完成练习。
4. 请自己选择并完成练习。
5. 请自己选择并完成练习。
6. 请自己选择并完成练习。
7. 请自己选择并完成练习。
8. 请自己选择并完成练习。

B.4 第 4 章：数据设计和性能模型

1. 代码清单 B.4.1 显示了分配左下三角数组的代码。假设数组索引为 C，左下角元素位于[0][0]。此外，矩阵必须是方阵。我们使用与代码清单 4-3 相同的代码，但每行 imax 的长度减少 1。注意三角数组的元素个数可通过 $j_{\max}*(i_{\max}+1)/2$ 来计算。

代码清单 B.4.1 三角矩阵分配

```
ExerciseB.4.1/malloc2Dtri.c
1 #include <stdlib.h>
2 #include "malloc2Dtri.h"
3
4 double **malloc2Dtri(int jmax, int imax)
5 {
6     double **x =
7         (double **)malloc(jmax*sizeof(double *) +
8                             jmax*(imax+1)/2*sizeof(double));
9     x[0] = (double *) (x + jmax);
10
11     for (int j = 1; j < jmax; j++, imax--) {
12         x[j] = x[j-1] + imax;
13     }
14     return(x);
15 }
16 }
```

首先为行指针和二维数组分配一块内存

现在在行指针之后为二维数组分配内存块的起始位置

每次迭代将 imax 减少 1

最后, 为每个行指针分配每个迭代位置所指向的内存

2. 假设我们在 Fortran 中将数组命名为 $x(j,i)$ 。如果我们创建一个宏 `#definex(j,i) x[i-1][j-1]`, 那么代码就可以使用 Fortran 数组表示法。代码清单 4-3 中的二维内存分配器可以通过交换 i 和 j 以及 $imax$ 和 $jmax$ 来使用。下面的代码清单显示了生成的代码。

代码清单 B.4.2 三角矩阵分配

```
Exercise4.2/malloc2Dfort.c
1 #include <stdlib.h>
2 #include "malloc2Dfort.h"
3
4 double **malloc2Dfort(int jmax, int imax)
5 {
6     double **x =
7         (double **)malloc(imax*sizeof(double *) +
8                             imax*jmax*sizeof(double));
9     x[0] = (double *) (x + imax);
10
11     for (int i = 1; i < imax; i++) {
12         x[i] = x[i-1] + jmax;
13     }
14     return(x);
15 }
16 }
```

首先为列指针和二维数组分配一块内存

现在在列指针之后为二维数组分配内存块的起始位置

最后, 为每个列指针分配要指向的内存位置

3. 我们用普通的数组索引和颜色名称检索数据:

```
#define VV=4
#define color(i,C) AOSOA[(i)/VV].C[(i)%4-1]
color(50,B)
```

4. 下图显示了去掉 if 语句的代码。从这个修改后的代码中, 性能模型计数如下所示:


```

Memops=2*NcNm+2*Nc=102MMemops
1: for all cells, C, up to Nc do
2:   ave ← 0.0
3:   for all material IDs, m, up to Nm do
4:     ave ← ave +  $\rho[C][m] * f[C][m]$       # 2NcNm loads ( $\rho, f$ )
                                           # 2NcNm flops (+, *)
5:   end for
6:    $\rho_{ave}[C] \leftarrow ave / V[C]$           # Nc stores ( $\rho_{ave}$ ), Nc loads (V)
                                           # Nc flops (/)
7: end for

```

性能模型=61.0 毫秒。此性能估计比使用 if 语句的版本略快。

5. 第 4.4 节中 ECM 模型的性能分析使用了一个 AVX-256 向量单元,它可以在 1 个周期内处理所有需要处理的浮点运算。AVX-512 仍然需要 1 个周期,但只有一半的向量单元处于忙碌状态,如果存在其他运算需求,则可以完成两倍的工作。因为计算操作时间 T_{OL} 保持在 1 个周期,所以性能根本不会改变。

B.5 第 5 章: 并行算法与模式

1. 碰撞操作的伪代码如下:

```

1. Bin particles into 1 mm spatial bins
2. For each bin
3.   For each particle, i, in the bin
4.     For all other particles, j, in this bin or adjacent bins
5.       if  $|P_i - P_j| < 1 \text{ mm}$ 
6.         compute collision

```

该操作在局部区域为 $O(N^2)$,但是随着网格变大,对于更大的区域不必计算粒子之间的距离,因此操作接近 $O(N)$ 。

2. 使用哈希函数对邮政编码的前三位数字,对州和地区进行编码,其余两个数字首先编码大的城镇,然后按字母顺序对其余部分进行编码。

3. 虽然是针对不同的问题域和规模开发的,但 map-reduce 算法中的 map 操作是一个哈希操作。所以都先完成一个哈希步骤,然后执行本地操作。空间哈希具有 bin 之间距离关系的概念,而 map-reduce 本质上没有。

4. 创建一个完美的空间哈希,bin 大小与最小 cell 相同,并将 cell 索引存储在 cell 底层的 bin 中。计算每个站点的 bin,并从 bin 获得 cell 索引。

B.6 第 6 章: 向量化: 免费的 flop

1. 请自己选择并完成练习。

2. 请自己选择并完成练习。

3. 在 `kahan_fog_vector.cpp` 当中, 将 `4s` 改为 `8s`, 将 `Vec4d` 改为 `Vec8d`。向 `CXXFLAGS` 添加 `mprefer-vector-width=512 -DMAX_VECTOR_SIZE=512`。更改后的代码和 `Makefile` 包含在本章的源代码中。

4. 对于 Intel 256 位向量单元, Intel 内在函数不起作用, 必须将其注释掉。然而, GCC 和 Fog 版本仍然有效。2017 年 Mac 笔记本电脑的计时结果显示了 Agner Fog 向量类库的优越性, `eight-wide` 向量比 `four-wide` 向量带来更好的结果。相比之下, `eight-wide` 向量的 GCC 实现比 `four-wide` 版本要慢。输出结果如下:

```
SETTINGS INFO -- ncells 1073741824 log 30
Initializing mesh with Leblanc problem, high values first
relative diff runtime      Description
      8.423e-09    1.461642    Serial sum
              0    3.283697    kahan sum with double double accumulator

4 wide vectors serial sum
      -3.356e-09    0.408654    Serial sum (OpenMP SIMD pragma)
      -3.356e-09    0.407457    Intel vector intrinsics Serial sum
      -3.356e-09    0.402928    GCC vector intrinsics Serial sum
      -3.356e-09    0.406626    Fog C++ vector class Serial sum
4 wide vectors kahan sum
              0    0.872013    Intel Vector intrinsics kahan sum
              0    0.873640    GCC vector extensions kahan sum
              0    0.872774    Fog C++ vector class kahan sum
8 wide vector serial sum
      -1.986e-09    1.467707    8 wide GCC vector intrinsic Serial sum
      -1.986e-09    0.586075    8 wide Fog C++ vector class Serial sum
8 wide vector kahan sum
      -1.388e-16    1.914804    8 wide GCC vector extensions kahan sum
      -1.388e-16    0.545128    8 wide Fog C++ vector class kahan sum
      -1.388e-16    0.687497    Agner C++ vector class kahan sum
```

B.7 第 7 章: 使用 OpenMP 实现并行计算

1. 转换为高级 OpenMP, 我们最终将得到以下代码清单中显示的代码, 其中只有一个 `pragma` 用来打开并行区域。

代码清单 B.7.1 高级 OpenMP

```
ExerciseB.7.1/vecadd.c
11 int main(int argc, char *argv[]){
12     #pragma omp parallel
13     {
14         double time_sum;
15         struct timespec tstart;
16         int thread_id = omp_get_thread_num();
17         int nthreads = omp_get_num_threads();
18         if (thread_id == 0){
19             printf("Running with %d thread(s)\n",nthreads);
```

```

20     }
21     int tbegin = ARRAY_SIZE * ( thread_id ) / nthreads;
22     int tend = ARRAY_SIZE * ( thread_id + 1 ) / nthreads;
23
24     for (int i=tbegin; i<tend; i++) {
25         a[i] = 1.0;
26         b[i] = 2.0;
27     }
28
29     if (thread_id == 0) cpu_timer_start(&tstart);
30     vector_add(c, a, b, ARRAY_SIZE);
31     if (thread_id == 0) {
32         time_sum += cpu_timer_stop(tstart);
33         printf("Runtime is %lf msecs\n", time_sum);
34     }
35 }
36 }
37
38 void vector_add(double *c, double *a, double *b, int n)
39 {
40     int thread_id = omp_get_thread_num();
41     int nthreads = omp_get_num_threads();
42     int tbegin = n * ( thread_id ) / nthreads;
43     int tend = n * ( thread_id + 1 ) / nthreads;
44     for (int i=tbegin; i < tend; i++){
45         c[i] = a[i] + b[i];
46     }
47 }

```

2. 约减例程使用 `reduction(max:xmax)` 子句，如下面的代码清单所示。

代码清单 B.7.2 OpenMP 最大约减

```

ExerciseB.7.2/max_reduction.c
1 #include <float.h>
2 double array_max(double* restrict var, int ncells)
3 {
4     double xmax = DBL_MIN;
5     #pragma omp parallel for reduction(max:xmax)
6     for (int i = 0; i < ncells; i++){
7         if (var[i] > xmax) xmax = var[i];
8     }
9 }

```

3. 在高级 OpenMP 中，我们手动划分数据。数据分解在代码清单 B.7.3 的第 6~9 行完成。线程 0 在第 13 行分配 `xmax_thread` 共享数据数组。第 18~22 行查找每个线程的最大值并将结果存储在 `xmax_thread` 数组中。然后，在第 26~30 行，通过一个线程在所有线程中找到最大值。

代码清单 B.7.3 高级 OpenMP

```

ExerciseB.7.3/max_reduction.c
1 #include <stdlib.h>
2 #include <float.h>
3 #include <omp.h>

```

```

4 double array_max(double* restrict var, int ncells)
5 {
6     int nthreads = omp_get_num_threads();
7     int thread_id = omp_get_thread_num();
8     int tbegin = ncells * ( thread_id )      / nthreads;
9     int tend   = ncells * ( thread_id + 1 )  / nthreads;
10    static double xmax;
11    static double *xmax_thread;
12    if (thread_id == 0){
13        xmax_thread = malloc(nthreads*sizeof(double));
14        xmax = DBL_MIN;
15    }
16    #pragma omp barrier
17
18    double xmax_thread_private = DBL_MIN;
19    for (int i = tbegin; i < tend; i++){
20        if (var[i] > xmax_thread_private) xmax_thread_private = var[i];
21    }
22    xmax_thread[thread_id] = xmax_thread_private;
23
24    #pragma omp barrier
25
26    if (thread_id == 0){
27        for (int tid=0; tid < nthreads; tid++){
28            if (xmax_thread[tid] > xmax) xmax = xmax_thread[tid];
29        }
30    }
31
32    #pragma omp barrier
33
34    if (thread_id == 0){
35        free(xmax_thread);
36    }
37    return(xmax);
38 }

```

B.8 第8章 MPI：并行骨干

1. 使用 `pack` 或数组缓冲区的版本安排发送，但在复制或发送数据之前返回。`MPI_Isend` 的标准规定：“在调用非阻塞发送操作后，发送方不应修改发送缓冲区的任何内容，直到发送完成。”`pack` 和数组版本在通信后，释放缓冲区。因此，这些版本可能会在复制缓冲区之前删除缓冲区，导致程序崩溃。因此为了安全起见，必须在删除缓冲区之前检查发送的状态。

2. 向量版本的程序，从原始数组发送数据，而不是复制。这比分配缓冲区的版本更安全，因为缓冲区将被释放。如果我们只阻塞接收，通信速度会更快。

3. 即便使用 `ghost cell` 交换的向量版本，我们也必须小心，不要修改仍在发送过程中的缓冲区。这种情况发生的概率可能很小，但它仍有可能发生。为了绝对安全，我们需要在更改数组之前检查发送是否已经完成。

4. 使用 `MPI_ANY_TAG` 作为标签参数，程序可以正常运行。并带来一些速度提升。通过使用

显式标签将会添加另一项检查，以确保接收到正确的消息。

5. 移除计时器中的 `barriers` 应该会提供更好的性能，并允许进程更独立地运行(异步)。不过，理解时间测量可能更困难。

6. 请自行完成。

7. 请自行完成。

B.9 第 9 章：GPU 架构及概念

1. 请自行完成。

2. 请自行完成。

3. 请自行完成。

B.10 第 10 章：GPU 编程模型

1. CPU 上的运行时间： $100\text{ ms} \times 1\,000\,000/16/1\,000 = 6\,250\text{ s}$

GPU 上运行的时间： $(5\text{ ms} + 5\text{ ms} + 5\text{ ms}) \times 1\,000\,000/1\,000 = 15\,000\text{ s}$

GPU 系统不会更快。所用时间是在 CPU 系统上的 2.5 倍。

2. 第 4 代 PCI 总线的速度是第 3 代 PCI 总线的两倍。

$(2.5\text{ ms} + 5\text{ ms} + 2.5\text{ ms}) \times 1\,000\,000/1\,000 = 10\,000\text{ s}$

第 5 代 PCI 总线的速度将是最初的第 3 代 PCI 总线的 4 倍。

$(1.25\text{ ms} + 5\text{ ms} + 1.25\text{ ms}) \times 1\,000\,000/1\,000 = 7\,500\text{ s}$

如果我们不必将结果传回，那么在 GPU 上的速度和在 CPU 上的速度一样快。

$(1.25\text{ ms} + 5\text{ ms}) \times 1\,000\,000/1\,000 = 6\,250\text{ s}$

3. NVIDIA GeForce GTX 1060 的内存大小为 6GiB。它带有一个 192 位宽总线和 8GHz 内存时钟的 GDDR5。

$$(6\text{ GiB}/2/4\text{ doubles}/8\text{bytes} \times 1024^3)^{1/3} = 465 \times 465 \times 465\text{ 3D mesh}$$

单精度：

$$(6\text{ GiB}/2/4\text{ floats}/4\text{bytes} \times 1024^3)^{1/3} = 586 \times 586 \times 586\text{ 3D mesh}$$

如果我们将计算域划分为这种 3D 网格，则分辨率将提高 25%。

B.11 第 11 章：基于指令的 GPU 编程

1. 请自行完成。

2. 请自行完成。

3. 本章中的性能结果来自于 NVIDIA V100 GPU

$3 \times 20\,000\,000 \times 8 \text{ bytes} / 0.586 \text{ ms} \times (1000 \text{ ms/s}) / (1\,000\,000\,000 \text{ bytes/GB}) = 819 \text{ GB/s}$

这比 NVIDIA P100 GPU 的 BabelStream benchmark 所显示的峰值高出 50%。

4. 这些数组只在 GPU 上使用，所以它们可以在那里分配并在最后删除。因此，将做如下改动：

```
13 #pragma omp target enter data map(alloc:a[0:nsize], b[0:nsize],
                                c[0:nsize])
36 #pragma omp target exit data map(delete:a[0:nsize], b[0:nsize],
                                c[0:nsize])
```

此更改的完整代码清单在本章示例中的 Stream_par7.c 中。

5. 我们只需要更改一个 pragma，如下所示。

代码清单 B.11.5 GPU 版本的 OpenMP

```
ExerciseB.11.5/mass_sum.c
1 #include "mass_sum.h"
2 #define REAL_CELL 1
3
4 double mass_sum(int ncells, int* restrict celltype,
5                 double* restrict H, double* restrict dx, double* restrict dy){
6     double summer = 0.0;
7     #pragma omp target teams distribute \
8         parallel for simd reduction(+:summer)
9     for (int ic=0; ic<ncells ; ic++) {
10         if (celltype[ic] == REAL_CELL) {
11             summer += H[ic]*dx[ic]*dy[ic];
12         }
13     }
14     return(summer);
15 }
```

6. 下面的代码清单显示了使用 OpenACC 查找最大半径的可能实现方式。

代码清单 B.11.6 OpenACC 版本的 Max Radius

```
ExerciseB.11.6/MaxRadius.c or Chapter11/OpenACC/MaxRadius/MaxRadius.c
1 #include <stdio.h>
2 #include <math.h>
3 #include <openacc.h>
4
5 int main(int argc, char *argv[]){
6     int ncells = 20000000;
7     double* restrict x = acc_malloc(ncells * sizeof(double));
8     double* restrict y = acc_malloc(ncells * sizeof(double));
9
10     double MaxRadius = -1.0e30;
11     #pragma acc parallel deviceptr(x, y)
12     {
13     #pragma acc loop
14         for (int ic=0; ic<ncells; ic++) {
15             x[ic] = (double)(ic+1);
16             y[ic] = (double)(ncells-ic);
```

```

17     }
18
19 #pragma acc loop reduction(max:MaxRadius)
20     for (int ic=0; ic<ncells ; ic++) {
21         double radius = sqrt(x[ic]*x[ic] + y[ic]*y[ic]);
22         if (radius > MaxRadius) MaxRadius = radius;
23     }
24 }
25 printf("Maximum Radius is %lf\n",MaxRadius);
26
27 acc_free(x);
28 acc_free(y);
29 }

```

B.12 第 12 章：GPU 语言：深入了解基础知识

1. 要获得固定内存，可以用 `cudaHostMalloc` 替换主机端内存分配中的 `malloc`，并且用 `cudaFreeHost` 替换空闲内存，如代码清单 B.12.1 所示。在代码清单中，我们只显示需要更改的部分行。将性能与第 12 章中 CUDA/StreamTriad 目录中的代码进行比较。使用固定内存时，数据传输时间至少要快两倍。

代码清单 B.12.1 固定内存版本的 stream triad

```

ExerciseB.12.1/StreamTriad.cu
31 // allocate host memory and initialize
32 double *a, *b, *c;
33 cudaMallocHost(&a,stream_array_size*sizeof(double));
34 cudaMallocHost(&b,stream_array_size*sizeof(double));
35 cudaMallocHost(&c,stream_array_size*sizeof(double));
36 < ... stream triad code ... >
86 cudaFreeHost(a);
87 cudaFreeHost(b);
88 cudaFreeHost(c);

```

2. 请自行完成。
3. 请自行完成。
4. 以下代码清单显示了在 GPU 上初始化了 a 和 b 数组的版本。

代码清单 B.12.4 在 SYCL 中初始化数组 a 和 b

```

14 // host data
15 vector<double> a(nsize);
16 vector<double> b(nsize);
17 vector<double> c(nsize);
18
19 t1 = chrono::high_resolution_clock::now();
20
21 Sycl::queue Queue(sycl::cpu_selector{});
22
23 const double scalar = 3.0;

```

```

24
25   Sycl::buffer<double,1> dev_a { a.data(), Sycl::range<1>(a.size()) };
26   Sycl::buffer<double,1> dev_b { b.data(), Sycl::range<1>(b.size()) };
27   Sycl::buffer<double,1> dev_c { c.data(), Sycl::range<1>(c.size()) };
28
29   Queue.submit([&](sycl::handler& CommandGroup) {
30
31       auto a =
32           dev_a.get_access<Sycl::access::mode::write>(CommandGroup);
33       auto b =
34           dev_b.get_access<Sycl::access::mode::write>(CommandGroup);
35       auto c =
36           dev_c.get_access<Sycl::access::mode::write>(CommandGroup);
37
38       CommandGroup.parallel_for<class StreamTriad>(
39           Sycl::range<1>{nsize}, [=] (Sycl::id<1> it) {
40               a[it] = 1.0;
41               b[it] = 2.0;
42               c[it] = -1.0;
43           });
44   });
45   Queue.wait();
46
47   Queue.submit([&](sycl::handler& CommandGroup) {
48
49       auto a = dev_a.get_access<Sycl::access::mode::read>(CommandGroup);
50       auto b = dev_b.get_access<Sycl::access::mode::read>(CommandGroup);
51       auto c =
52           dev_c.get_access<Sycl::access::mode::write>(CommandGroup);
53
54       CommandGroup.parallel_for<class StreamTriad>(
55           Sycl::range<1>{nsize}, [=] (Sycl::id<1> it) {
56               c[it] = a[it] + scalar * b[it];
57           });
58   });
59   Queue.wait();
60
61   t2 = chrono::high_resolution_clock::now();

```

5. 初始化循环需要完成以下代码清单中所示的更改。然后使用与第 12.5.2 节中相同的方式构建和运行 stream triad 代码。

代码清单 B.12.5 将 Raja 添加到 stream triad 的初始化循环中

```

ExerciseB.12.5/StreamTriad.cc
19 RAJA::forall<RAJA::omp_parallel_for_exec>(RAJA::RangeSegment(0,
20     nsize), [=] (int i) {
21     a[i] = 1.0;
22     b[i] = 2.0;
23 });

```

通过这些更改，与第 12.5.2 节中的原始版本相比，运行时间从大约 6.59 毫秒降到 1.67 毫秒。

B.13 第 13 章：GPU 配置分析及工具

1. 请自行完成。
2. 请自行完成。
3. 请自行完成。

B.14 第 14 章：关联性：与 kernel 休战

1. 使用 `lstopo` 工具生成架构图。
2. 请自行完成。
3. 请自行完成。
4. 请自行完成。
5. 请自行完成。
6. 请自行完成。
7. 请自行完成。

B.15 第 15 章 批处理调度器：为混乱带来秩序

1. 请自行完成。
2. 要插入预处理步骤，我们需要插入另一段条件代码，如下面的第 31~36 行所示，然后使用 `PREPROCESS_DONE` 文件来指示预处理已经完成。

代码清单 B.15.2a 插入预处理步骤，然后自动重启

```
ExerciseB.15.2/Preprocess_then_restart.sh
1 #!/bin/sh
2 #SBATCH -N 1
3 #SBATCH -n 4
4 #SBATCH --signal=23@160
5 #SBATCH -t 00:08:00
6
7 # Do not place bash commands before the last SBATCH directive
8 # Behavior can be unreliable
9
10 NUM_CPUS=4
11 OUTPUT_FILE=run.out
12 EXEC_NAME=./testapp
13 MAX_RESTARTS=4
14
15 if [ -z ${COUNT} ]; then
16     export COUNT=0
17 fi
18
19 ((COUNT++))
```

```

20 echo "Restart COUNT is ${COUNT}"
21
22 if [ ! -e DONE ]; then
23     if [ -e RESTART ]; then
24         echo "=== Restarting ${EXEC_NAME} ==="           >> ${OUTPUT_FILE}
25         cycle=`cat RESTART`
26         rm -f RESTART
27     elif [ -e PREPROCESS_DONE ]; then
28         echo "=== Starting problem ==="                 >> ${OUTPUT_FILE}
29         cycle=""
30     else
31         echo "=== Preprocessing data for problem ===" >> ${OUTPUT_FILE}
32         mpirun -n ${NUM_CPUS} ./preprocess_data &>> ${OUTPUT_FILE}
33         date > PREPROCESS_DONE
34         sbatch \
            --dependency=afterok:${SLURM_JOB_ID} \      | 在预处理后提交
            <preprocess_then_restart.sh                | 第一个计算作业
35     exit
36 fi
37
38 echo "=== Submitting restart script ==="             >> ${OUTPUT_FILE}
39 sbatch \
    --dependency=afterok:${SLURM_JOB_ID} \      | 提交重启作业
    <preprocess_then_restart.sh
40
41 mpirun -n ${NUM_CPUS} ${EXEC_NAME} ${cycle}      &>> ${OUTPUT_FILE}
42 echo "Finished mpirun"                          >> ${OUTPUT_FILE}
43
44 if [ ${COUNT} -ge ${MAX_RESTARTS} ]; then
45     echo "=== Reached maximum number of restarts ===" >> ${OUTPUT_FILE}
46     date > DONE
47 fi
48 fi

```

通常，预处理步骤需要不同数量的处理器。这种情况下，可使用一个单独的批处理脚本进行预处理，如下所示。

代码清单 B.15.2b 更少的预处理步骤，然后自动重启

```

ExerciseB.15.2/Preprocess_batch.sh
1 #!/bin/sh
2 #SBATCH -N 1
3 #SBATCH -n 1
4 #SBATCH -t 01:00:00
5
6
7 sbatch --dependency=afterok:${SLURM_JOB_ID} <batch_restart.sh
8
9
10 mpirun -n 4 ./preprocess &> preprocess.out

```

3. 更改 Slurm 批处理脚本，从而检查命令的状态，并删除仿真数据库。有几种不同的方法可以进行清理。这里提供 3 个供你参考。代码清单 B.15.3a 和 b 中的前两个代码使用来自 `mpirun` 命令的退出代码。

代码清单 B15.3.a OpenACC 版本的 Max Radius

```

ExerciseB.15.3/batch_simple_error.sh
1 #!/bin/sh
2 #SBATCH -N 1
3 #SBATCH -n 4
4 #SBATCH -t 01:00:00
5 #SBATCH -t 01:00:00
6
7 mpirun -n 4 ./testapp &> run.out || \
    rm -f simulation_database

```

||符号对非零状态
值执行命令

代码清单 B15.3.b OpenACC 版本的 Max Radius

```

ExerciseB.15.3/batch_simple_error.sh
1 #!/bin/sh
2 #SBATCH -N 1
3 #SBATCH -n 4
4 #SBATCH -t 01:00:00
5 #SBATCH -t 01:00:00
6
7 mpirun -n 4 ./testapp &> run.out
8 STATUS=$?
9 if [ ${STATUS} != "0" ]; then
10     rm -f simulation_database
11 fi

```

代码清单 B.15.3.c 中的第三个版本通过依赖标志，使用批处理作业的状态条件来调用一个清理作业。处理的错误类型与前两种方法不同。

代码清单 B15.3.c OpenACC 版本的 Max Radius

```

ExerciseB.15.3/batch.sh
1 #!/bin/sh
2 #SBATCH -N 1
3 #SBATCH -n 4
4 #SBATCH -t 01:00:00
5 #SBATCH -t 01:00:00
6
7 sbatch --dependency=afternotok:${SLURM_JOB_ID} <batch_cleanup.sh
8
9
10 mpirun -n 4 ./testapp &> run.out

```

```

ExerciseB.15.3/batch_cleanup.sh
1 #!/bin/sh
2 #SBATCH -N 1
3 #SBATCH -n 1
4 #SBATCH -t 00:10:00
5 #SBATCH -t 00:10:00
6 rm -f simulation_database

```

B.16 第 16 章：并行环境的文件操作

1. 请自行完成。

2. 请自行完成。
3. 请自行完成。

B.17 第 17 章：用于编写优质代码的工具和资源

1. 请自行完成。
2. 请自行完成。
3. 请自行完成。
4. 请自行完成。
5. 请自行完成。